Week 4 Interview Question

Observability, Performance & Auto-Healing

Objective: Build a complete observability stack → understand performance bottlenecks → build auto-healing & scaling systems.

Section 1: Prometheus Fundamentals

1. Explain how Prometheus scrapes metrics. What is the role of service discovery in Kubernetes.

2. What is the difference between counters, gauges, histograms, and summaries. Give examples for each.

3. How does Prometheus store data internally and why is it optimized for time-series.

4. What causes Prometheus to show “stale” values and how do you troubleshoot them.

5. What happens if a target is down during scraping. How does Prometheus reflect it in queries.

6. Explain how recording rules work. Why do production setups depend heavily on them.

7. What is federation in Prometheus and when is it used.

8. How do you troubleshoot high cardinality metrics. What impact can they have on Prometheus performance.

9. How do you identify whether Prometheus is overloaded. What metrics indicate overload.

10. Explain step-by-step how you would check if a metric is being scraped, stored, and queried correctly.

Section 2: PromQL Scenarios

1. Write a query to calculate node CPU usage across all nodes.
2. Write a query to detect pods restarting more than 5 times in 10 minutes.
3. How do rate() and irate() differ and when do you prefer one over the other.
4. Explain histogram_quantile and how p90 or p99 latency is calculated.
5. Given a high error rate alert firing, walk through your debugging approach using PromQL.

Section 3: Grafana and Dashboarding

1. What are best practices for building production dashboards.
2. How do you design a dashboard that helps detect saturation, errors, and latency.
3. What is the difference between dashboard variables and labels.
4. How do you troubleshoot missing data in Grafana panels.
5. Explain how Grafana connects to Prometheus and Loki. What common issues appear when configuring them.

Section 4: Logging: Loki and FluentBit

1. Explain Loki's architecture and how it differs from Elasticsearch.
2. What is label cardinality and how does it affect log storage and performance.
3. How does FluentBit tail logs and ship them to Loki. What happens when FluentBit restarts.
4. Describe how to troubleshoot a situation where logs are not appearing in Grafana Explore.
5. When searching logs, what is the difference between using grep-style queries and LogQL filter expressions.

6. Explain how retention, compaction, and chunk storage works in Loki.

Section 5: Distributed Tracing and OpenTelemetry

1. What problem does distributed tracing solve in microservice architectures.
2. Explain spans, trace IDs, parent-child relationships, and context propagation.
3. How does the OpenTelemetry Collector process and export spans.
4. How would you debug missing spans for a microservice.
5. In a trace, how do you identify the slowest segment and the root cause of latency.
6. Difference between metrics, logs, and traces in terms of purpose and storage.

Section 6: Kubernetes Autoscaling (HPA, VPA, KEDA)

1. Explain how the Horizontal Pod Autoscaler works and what metrics it relies on.
2. How does the VPA recommender work. What are the three VPA modes and their use cases.
3. What happens when both HPA and VPA act on the same deployment.
4. How do you debug an HPA that is not scaling even though traffic is high.
5. Explain event-driven scaling with KEDA and how it differs from HPA.
6. Give real examples of scaling based on CPU, custom metrics, and external metrics.
7. What is a good strategy for testing autoscaling in staging.

Section 7: Alerting and SLO Design

1. Explain the difference between SLIs, SLOs, and SLAs.
2. What is an error budget and how does it influence operational decisions.
3. How would you design an alert rule that avoids noise and focuses on symptoms, not causes.
4. Explain how Alertmanager groups, routes, and silences alerts.
5. What is a multi-window, multi-burn-rate alert and why is it recommended by Google SRE.
6. How do you choose thresholds for alerts. What is the danger of alerting on raw metrics.
7. How do you audit an existing alerting system for reliability and noise.

Section 8: Troubleshooting and Real Scenarios

1. Prometheus shows gaps in data. What are the top five things you check.
2. Grafana panel shows “No Data”. Explain your end-to-end troubleshooting steps.
3. Logs for one workload are missing in Loki. What are the possible root causes.
4. Traces are partially appearing or missing children spans. How do you debug.
5. HPA is scaling too aggressively. What do you investigate.
6. System is slow but dashboards look normal. How do you perform deep-dive debugging using metrics, logs, and traces.
7. A node becomes NotReady. Which observability tools help explain why.

Section 9: Advanced Concepts

1. Explain the RED and USE monitoring models and when to apply them.
2. What is the difference between blackbox and whitebox monitoring. Give examples.
3. Explain service-level indicators for latency, saturation, and error rate for a backend service.
4. How would you benchmark the performance of your observability stack itself.
5. Describe a high-level design for a multi-cluster observability setup.

Section 10: Practical Hands-On Questions

1. Show how to write a PromQL alert that fires when CPU usage is above 90% for 5 minutes.
2. Show how to build a node dashboard with CPU, memory, disk, and network metrics.
3. Show how to query logs for a pod restarting continuously.
4. Show how to visualize p99 latency for an API using histogram metrics.
5. Show how to simulate traffic and validate HPA scaling.